

COS710

Assignment 1

Stefan Jansen van Rensburg

u22550055

## Using Genetic Programming (GP) to predict the electricity load for a day at a particular time.

### Introduction:

Electricity load prediction is an important problem that affects distribution of and generation of electricity. Below is a detailed explanation of my creation of a genetic programming algorithm that attempts to solve the problem.

### Problem Description:

The aim of this GP is to predict the electricity load for a particular day and time based on factors from the  $n$  previous days. The dataset contains residential energy data from the UK spanning close to a five-year timespan from the 31st of December 2014 at 23:00 to the 30th of September 2020 at 23:45 in 15 minute intervals for a total of 201604 records. Each record contains the utc timestamp, electricity load, residential electricity price, residential solar generation, residential wind generation, temperature and relative humidity. (cite dataset) The units of measurement for these values are not important for the creation of the genetic programming algorithm except for electricity load which is measured in kilowatts (kW).

The dataset is divided into the training set and the testing set. Two thirds of the data is randomly selected to be the training set, and the remaining data is the testing set. Before the data is used, the titles of the data are removed.

From the dataset the input or terminal values for the GP algorithm are largely the dataset values with minor differences. Since the problem is time-series based, the terminal values rely on a parameter being set, which is called numPreviousDays which will be represented by  $n$ . As such, the terminal set includes:

electricity\_load-1, electricity\_load-2, ..., electricity\_load-n

residential\_electricity\_price-1, residential\_electricity\_price-2, ...,  
residential\_electricity\_price-n

residential\_solar\_generation-1, residential\_solar\_generation-2, ...,  
residential\_solar\_generation-n

residential\_wind\_generation-1, residential\_wind\_generation-2, ...,  
residential\_wind\_generation-n

temperature-1, temperature-2, ..., temperature-n

relative\_humidity-1, relative\_humidity-2, ..., relative\_humidity-n

hour-1, hour-2, ..., hour-n

day-2, day-2, ..., day-n

Hour is taken from the utc\_timestamp. The recurrence of hour is an unfortunate result of how I created the terminals.

Day is calculated using Zeller's Congruence, which is a formula which uses the year, month and day to calculate the day of the week. The reason I decided to use Zeller's Congruence is because I realised that I live in a weekly cycle and most people do as well, so I was hoping that this formula would help my GP algorithm to capture that.

The numerical values that get assigned to the terminals are just the raw values found in the dataset except for hour and day as explained above.

The function set simply includes  
"ADD", "SUBTRACT", "MULTIPLY", and "DIVIDE"

## Genetic Programming Setup

The Genetic Programming Setup includes 5 classes and is based on a tree structure:

Node

Tree

TrainingCase

GP

Main

### Node:

Nodes have four constructors with only three of them being important.

Nodes can be created to either randomly be a random terminal node or a random function node, a random terminal node, or a random function node.

Nodes have a type (terminal or function), value (string value to tell what it represents) and a numerical value that gets assigned to it based on its value.

The numerical value is used for evaluation. In the case of function nodes, the value holds a string to represent its function. When it is evaluated, the string is interpreted to perform its mathematical function. The numerical value holds what it has calculated based on its children.

### Tree

When a tree is evaluated, the head node recursively evaluates, so if it is a function node it will evaluate its own value last. The head node's numerical value is compared to the actual value for fitness and will be explained later.

## TrainingCase

A training case is a random selection of a date and  $n$  (numPreviousDays) previous values. The previous values are 94 records away each to account for each day to be at the same time since all the records are recorded at 15 minute intervals. During training, a tree is evaluated on a certain sized batch of training cases. If the batch size parameter is not set, the default is set to 16 training cases per batch. Note, since the batches are random, fitness calculated from them is noisy, but I believe still gives a good measure of fitness.

The training case class is also used for the testing set, but is then just made of previously guaranteed unseen values.

## Initial Population Generation

I have decided to use Ranked Half and Half for initial population generation, so for each depth from 1 (head and two children) onwards half of the population is generated from “Grow”, where each node is randomly generated as either a function or a terminal based on function probability set in the GP parameters. The other half is generated from “Full” where every node is a random function node except the nodes at max depth. They are random terminals. I decided to use Ranked half and half so that small trees without bloat that produce accurate results can be selected as well as complex trees that may have an interesting complex result.

## Fitness Evaluation

For fitness evaluation, mean square error (MSE ) is used.

Each tree is evaluated against a training case and  $(\text{predicted} - \text{actual})^2$  is calculated. This happens for a batch of training cases and the sum of those calculations divided by the number of training cases in a batch to give the mean square error. Any NaN, Infinite or negative numbers are replaced by a large constant so that super extreme values do not skew the data collected too much. It also makes it very unlikely that those individuals will be selected. Division by zero is skipped rather than returning infinity.

## Selection Method

Tournament selection is used. A random draw happens of the size set in the parameters. The individual that is most fit (lowest MSE) is selected.

## Genetic Operators

Mutation, Crossover and Reproduction are used.

For mutation, the selected tree has a random node replaced by a random subtree. The probability that the subtree will be a terminal node is 1 - function probability (set in parameters). The mutated tree is then added to the next generation.

For Crossover, two subtrees are selected. Then, two random nodes are selected. The randomly selected nodes can be subtrees or leaf nodes. Then those two randomly selected nodes are swapped. The two trees created by this process are put into the next generation. This can only happen if the next generation is 2 trees or more away from being complete.

For reproduction, the selected tree is simply cloned into the next generation.

To fill the next generation, a random operator is chosen based on their probabilities set in the parameters, and then the above is performed.

## Experimental Setup

The following parameter values were used:

NumPreviousDays:7

Seed:83839

PopulationSize:700

InitialPopulationGeneration:HalfAndHalf

MaxDepth:9

FunctionProbability:82

TournamentSize:5

MutationRate:10

CrossoverRate:87

ReproductionRate:3

SelectionType:"tournament"

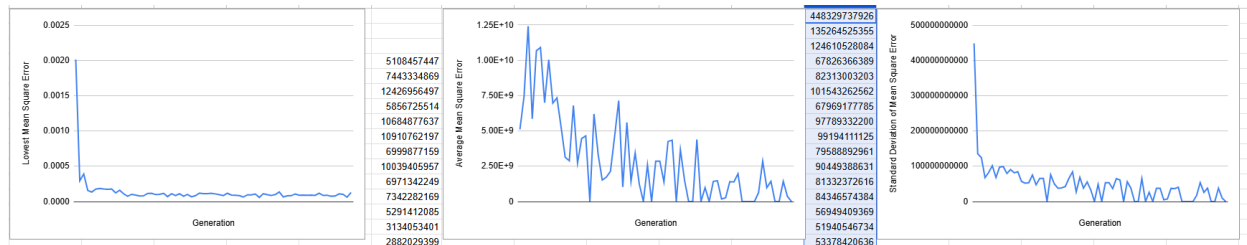
Runs:80

## Computer Specifications:

AMD Ryzen 7 7735HS with Radeon Graphics (3.20 GHz)

Ram: 64GB

Run 1: seed 83839



Best prediction | predicted=0.08600003 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

Avg Mean Square Error = 17324828.30936141

Standard Deviation of Mean Square Error = 458035828.99599135

Lowest Mean Square Error = 0.00008450

Run 2: 83840

Runtime: 16457 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=12809583457.55538000 | stdMSE=104928886571.86047000 |

lowestMSE=0.00007669

Run3: 83841

Runtime: 16856 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=14549848236.11151700 | stdMSE=109711242130.14662000 |

lowestMSE=0.00007662

Run4: 83842

Runtime: 18914 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=16687962102.25243400 | stdMSE=116383396624.01003000 |  
lowestMSE=0.00067087

Run5: 83843

Runtime: 20483 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=14767924487.38352600 | stdMSE=98254087315.07780000 |  
lowestMSE=0.00010769

Run6:83844

Runtime: 18483 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=11227989979.30976700 | stdMSE=97171176642.03633000 |  
lowestMSE=0.00009125

Run7:83845

Runtime: 17499 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=14987598427.56025100 | stdMSE=104309302490.96230000 |  
lowestMSE=0.00005894

Run8:83846

Runtime: 17775 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=8364746646.08677300 | stdMSE=81178904376.72755000 |  
lowestMSE=0.00007956

Run9:83847

Runtime: 1059183 ms

Best prediction | predicted=0.08599982 | actual=0.07100000 |  
mse=0.00022499

Last generation stats:

predicted=0.08599959 | actual=0.07100000 | mse=0.00022499  
avgMSE=14.42832229 | stdMSE=318.30206210 | lowestMSE=0.00010306

Run 10: 83848

Runtime: Runtime: 19479 ms

Best prediction | predicted=0.08600000 | actual=0.07100000 |  
mse=0.00022500

Last generation stats:

avgMSE=6810571054.14294300 | stdMSE=64617967091.37419000 |  
lowestMSE=0.00008775

Note, the reason for the difference between lowestMSE and the best tree's MSE is because they are evaluated with different testing cases.



## Discussion

The largest differences in run time come from testing the final tree. The final tree is almost always either a single node tree being electricity\_load-7 or is a very large tree. Median might have been a better measure for generational fitness improvement as average and standard deviation are largely affected by degenerate trees.